



Conceptos básicos de programación

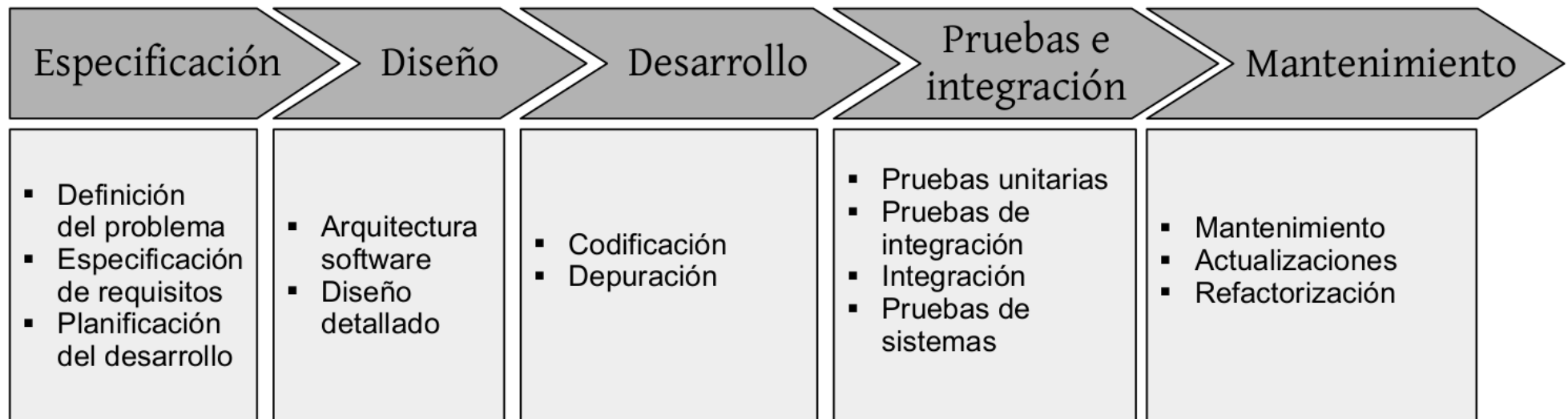
Un breve repaso...

Algoritmo

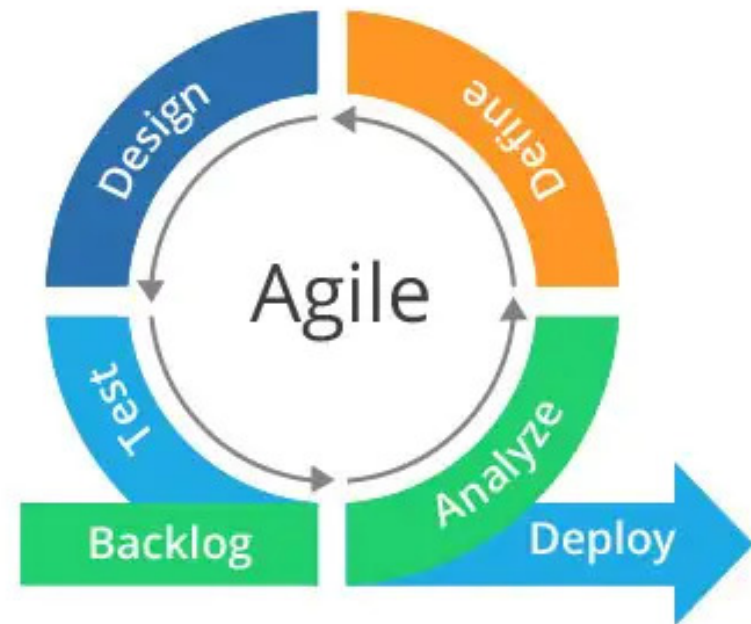


Fases del desarrollo de software

Metodología clásica (cascada)



Waterfall vs. Agile



Documentación de código

¿Cuál es mejor?

```
// tax rate  
x = 21
```

```
taxRate = 21
```

Documentación de código

¿Cuál es mejor?

```
/**
 * Muestra "Hola"
 *
 * @author Luis J. Sánchez
 */
public class HolaMundo {
    public static void
        main(String[] args) {
        System.out.println("Hola");
    }
}
```

```
/**
 * Muestra "Hola"
 */
public class HolaMundo {
    public static void
        main(String[] args) {
        System.out.println("Hola");
    }
}
```

La importancia de la simplicidad



VS



"Pop Bottle Opener" Rube Goldberg Machine

<https://www.youtube.com/watch?v=tPFUCC1VL8o>

KISS principle

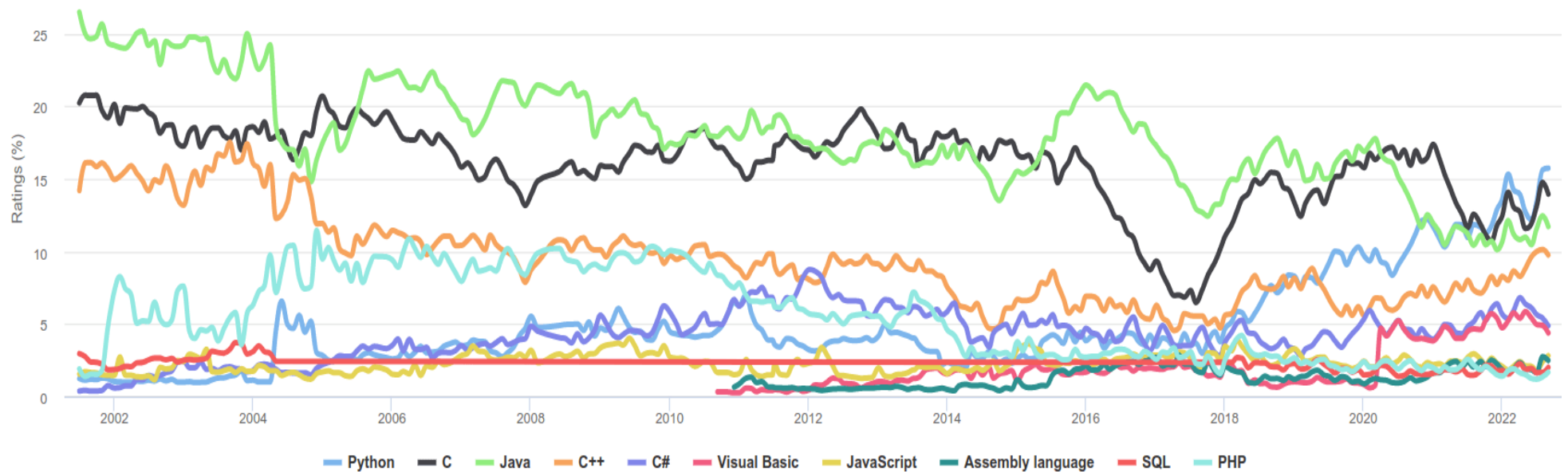


Keep it simple, stupid!
Keep it small and simple ✓

Lenguajes de programación

ADA	BASIC	C	C++
COBOL	COFFEScript	DART	DELPHI
ERLANG	SMALLTALK	FORTRAN	GO
JULIA	JAVAScript	KOTLIN	LISP
PIET	OBJETIVE-C	PEARL	PHP
PYTHON	BRAINFUCK	RUBY	RUST
TCL	Typescript	UNICORN	R
CLOJURE	ELIXIR	HASKELL	MATLAB
SCALA	MODULA-2	C#	EIFFEL
GROOVY	LOGO	PROLOG	F
LOLCODE	PIZZA	AWK	OBERON

Índice TIOBE



<https://www.tiobe.com/tiobe-index/>

Lenguajes de programación:

→ Interpretados

→ Compilados

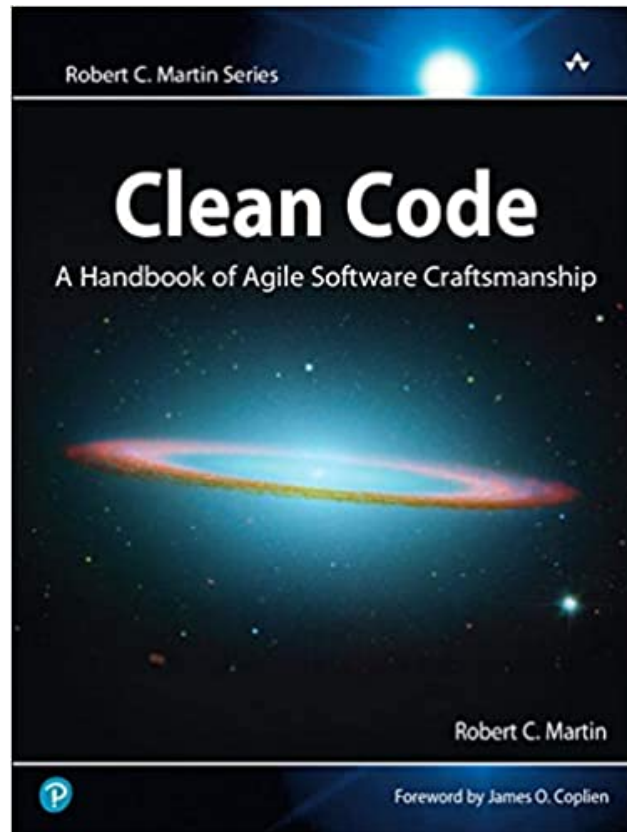
Pregunta:

**¿Qué hace un programador la
mayor parte del tiempo?**

Respuesta: Leer



El código debe ser, ante todo, legible
Código limpio (clean code)



A Handbook of Agile Software Craftsmanship

General rules	Names rules	Use as explanation of intent	Small number of instance variables
Follow standard conventions	Choose descriptive and unambiguous names	Use as clarification of code	Base class should know nothing about their derivatives
Keep it simple stupid. Simpler is always better Reduce complexity as much as possible	Make meaningful distinction	Use as warning of consequences	Better to have many functions than to pass some code into a function to select a behavior
Boy scout rule. Leave the campground cleaner than you found it	Use pronounceable names	Source code structure	Prefer non-static methods to static methods
Always find root cause Always look for the root cause of a problem	Use searchable names	Separate concepts vertically	Tests
Design rules	Replace magic numbers with named constants	Related code should appear vertically dense	One assert per test
Keep configurable data at high levels	Avoid encodings. Don't append prefixes or type information	Declare variables close to their usage	Readable
Prefer polymorphism to if/else or switch/case	Functions rules	Dependent functions should be close	Fast
Separate multi-threading code	Small	Similar functions should be close	Independent
Prevent over-configurability	Do one thing	Place functions in the downward direction	Repeatable
Use dependency injection	Use descriptive names	Keep lines short	Code smells
Follow Law of Demeter A class should know only its direct dependencies	Prefer fewer arguments	Don't use horizontal alignment	Rigidity. The software is difficult to change. A small change causes a cascade of subsequent changes
	Have no side effects	Use white space to associate related things and disassociate weakly related	Fragility. The software breaks in many places due to a single change
Understandability tips	Don't use flag arguments. Split method into several independent methods that can be called from the client without the flag	Don't break indentation	Immobility. You cannot reuse parts of the code in other projects because of involved risks and high effort
Be consistent If you do something a certain way, do all similar things in the same way	Comments rules	Objects and data structures	Needless. Complexity
Use explanatory variables	Always try to explain yourself in code	Hide internal structure	Needless. Repetition
Encapsulate boundary conditions. Boundary conditions are hard to keep track of Put the processing for them in one place	Don't be redundant	Prefer data structures	Opacity. The code is hard to understand
Prefer dedicated value objects to primitive type	Don't add obvious noise	Avoid hybrids structures (half object and half data)	'Clean code' by Robert C. Martin summary by Wojtek Lukaszuk
Avoid logical dependency. Don't write methods which works correctly depending on something else in the same class	Don't use closing brace comments	Should be small	
Avoid negative conditionals	Don't comment out code Just remove	Do one thing	

Patrones de diseño

Soluciones probadas a problemas
comunes en el desarrollo de software.

➔ Singleton

➔ Factory Method

➔ Observer

➔ Decorator

Singleton

Garantiza que una clase tenga solo una instancia y proporciona un punto de acceso global a esa instancia.

En un sistema de gestión de préstamos de una biblioteca, hay muchos libros, pero solo una biblioteca. La clase **Biblioteca** sigue el patrón **singleton**, ya que solo existe una biblioteca en todo el sistema, aunque haya muchos libros. Además, la clase **Biblioteca** es un punto de acceso único a toda la información y los procesos de la biblioteca.

Factory Method

Permite crear objetos sin especificar la clase exacta de objeto que se va a crear.

En un restaurante donde el menú cambia dependiendo del día, el **Factory Method** sería como el chef que, según el día, decide qué plato preparar. Así, el cliente pide comida sin tener que saber qué ingredientes específicos usa el chef o cómo se prepara, simplemente el chef "fabrica" el plato adecuado para ese día.

Observer

Permite que un objeto notifique a otros objetos cuando algo cambia sin necesidad de que estén estrechamente acoplados.

En una tienda on-line, cada vez que hay una nueva oferta, el sistema notifica automáticamente a todos los usuarios que están suscritos. El patrón **Observer** permite que, cuando hay un cambio (una nueva oferta), todos los usuarios suscritos (observadores) reciban la notificación sin que la tienda tenga que contactar uno por uno con ellos.

Decorator

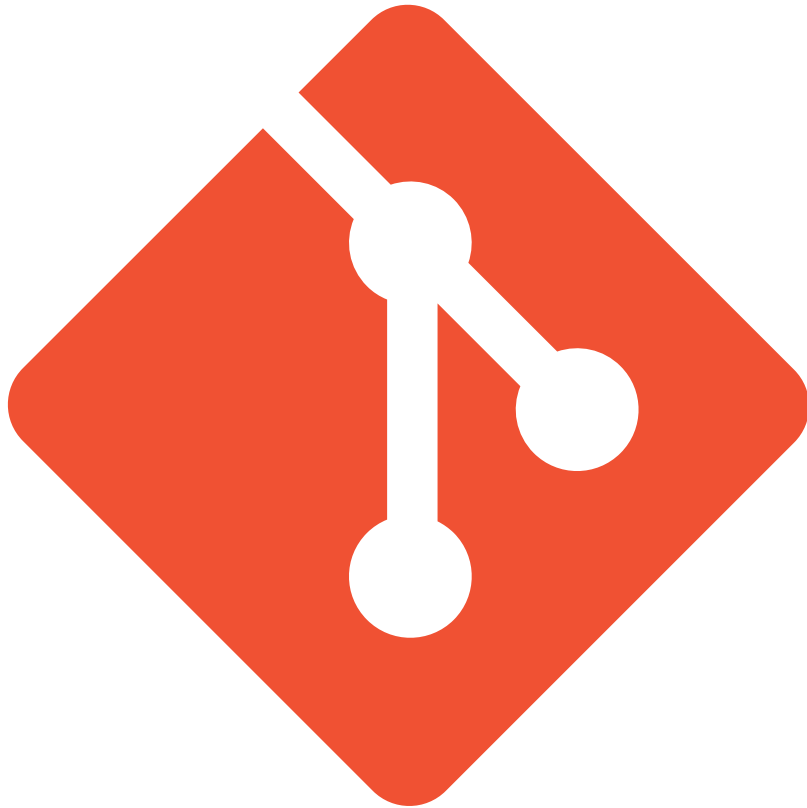
Permite añadir funcionalidades a un objeto de manera flexible sin modificar su estructura interna.

En una tienda de ropa hay chaquetas. Si alguien decide añadir un pañuelo o un sombrero, no se está modificando la chaqueta, solo añadiendo accesorios. El patrón Decorator permite añadir funcionalidades o características adicionales a un elemento (en este caso, a la chaqueta) sin cambiar su estructura original.

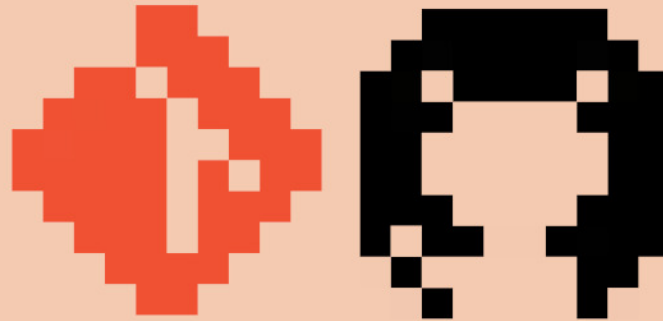


<https://refactoring.guru/es/design-patterns>

Control de versiones



{ LUIS JOSÉ SÁNCHEZ }



Git y GitHub

Guía de Supervivencia

[Lo básico para mantener tu código bajo control]

<https://leanpub.com/gitygithub>

Comunidad



Créditos de las imágenes

Wikimedia Commons

Pixabay

Pexels

CEP de Cádiz

Refactoring Guru